EventOS: modelagem e implantação de um banco de dados para eventos

José Carlos Justiniano de Lima Filho, Arthur Felipe Cairo Guedes, Shênio de Souza Alves e Natan Barreto de Souza Alves

Universidade Federal da Bahia
MATA60 - Banco de Dados | Marco 1 | Grupo 4

Abstract

This report presents EventOS, a PostgreSQL database for managing events, editions, participants, a global category catalogue and registrations. Its scope includes ten selected functional requirements and an optional extension for the promotion channel. The conceptual model follows Peter Chen notation and maps to five tables with 28 columns. A deterministic synthetic load specifies 23,502 rows. The report describes the relational mapping, integrity mechanisms, controlled transactions and available evidence, and provides SQL sources and reproduction instructions. Declarative constraints, operational rules and evidence limitations are distinguished throughout the report.

Resumo

Este relatório apresenta o EventOS, banco PostgreSQL para administrar eventos, edições, participantes, categorias globais e inscrições. O recorte reúne dez requisitos funcionais selecionados e uma extensão opcional para canal de divulgação. O modelo conceitual utiliza a notação Peter Chen e origina cinco tabelas com 28 colunas. A carga sintética determinística prevê 23.502 linhas. São descritos a tradução relacional, os mecanismos de integridade, as transações controladas e as evidências disponíveis, com códigos SQL e tutorial de reprodução. O texto distingue restrições declarativas, regras operacionais e limites de comprovação.

1. Descritivo do Projeto

1.1. Objetivo e recorte dos requisitos

O EventOS organiza o cadastro de eventos e suas realizações, reutiliza participantes entre edições e administra a inscrição e sua quitação atual. O gestor consulta quantidades por evento, edição, categoria e situação, percentuais de inscrições pagas, criações diárias e acumuladas, recebimento bruto atual e distribuição por canal de divulgação. A entrega utiliza dados fictícios e scripts SQL, sem site, API ou integração financeira.

O projeto original foi delimitado em um núcleo gerencial com dez requisitos selecionados: RF1 e RF28 mantidos e oito refinados. Os outros vinte requisitos foram excluídos com justificativas no documento de escopo. A Tabela 1.1 resume o texto vigente; os refinamentos representam reduções expressas do escopo original, especialmente quanto a categorias, lotes, situações e pagamentos.

Tabela 1.1. Requisitos funcionais selecionados e seu alcance.

Requisito	Classificação	Alcance na entrega
RF1	Mantido	Cadastrar eventos e suas diversas edições.
RF3	Refinado	Reutilizar participantes com nome, e-mail e CPF facultativo, exclusivo quando informado.
RF5	Refinado	Manter categorias e preços vigentes em catálogo global, utilizável por qualquer edição.
RF6	Refinado	Adotar uma janela de inscrições por edição e copiar o preço positivo da categoria para o contrato.
RF9	Refinado	Manter apenas a situação atual: PENDENTE, PAGA ou CANCELADA.
RF13	Refinado	Reconhecer administrativamente a quitação integral, sem registrar tentativas, forma ou data de pagamento.
RF26	Refinado	Produzir indicadores atuais de volume, situação, recebimento bruto e divulgação.
RF27	Refinado	Preservar contrato, data e vínculos da inscrição; conservar inscrições canceladas.
RF28	Mantido	Aplicar integridade e distinguir constraints, protocolo, privilégios e diagnósticos.
RF30	Refinado	Minimizar dados e separar consulta agregada de acesso operacional.

Além desses requisitos, a extensão **RE1** registra um único canal de divulgação, opcional, informado na criação da inscrição: `SITE_EVENTO`, `REDE_SOCIAL`, `EMAIL`, `INDICACAO` ou `OUTRO`. `NULL` significa ausência de informação. A extensão permite comparar divulgação por edição sem cadastrar campanhas ou integrar plataformas. As demais propostas individuais foram avaliadas e não integram o modelo implantado.

1.2. Minimundo e regras de negócio

Um evento pode possuir várias edições. Cada edição pertence a exatamente um evento e tem designação, período de realização, modalidade e uma janela única de inscrições, com limites inclusivos. As modalidades são `PRESENCIAL`, `HIBRIDA` e `REMOTA`; o local é obrigatório nas duas primeiras e deve estar ausente na modalidade remota. O início da realização não pode ultrapassar o fim, e abertura e fechamento das inscrições devem respeitar `abertura <= fechamento <= data_fim`.

O participante possui identificador, nome e e-mail; o CPF é opcional. Nomes e e-mails podem se repetir. O CPF informado deve conter onze dígitos e ser exclusivo, sem validação de autenticidade. Sem CPF, o esquema não permite deduplicar com segurança pessoas reais. O mesmo cadastro pode participar de várias edições, mas só pode possuir uma inscrição em cada edição, inclusive se a inscrição estiver cancelada.

A categoria é uma classificação administrativa global, com nome único, descrição facultativa e preço vigente positivo em reais. Todas as categorias podem ser utilizadas em qualquer edição; não há catálogo nem preço independente por edição. A inscrição relaciona diretamente um participante, uma edição e uma categoria. Na criação pelo protocolo, o preço vigente é copiado para `valor_contratado`. Reajustes posteriores da categoria não alteram contratos antigos.

Pelo protocolo operacional, a inscrição nasce `PENDENTE` e pode passar a `PAGA` ou `CANCELADA`, ambos terminais. Não há reabertura, cancelamento de inscrição paga ou troca de participante, categoria e edição. A quitação de uma pendente pode ocorrer após o fechamento das inscrições. A situação `PAGA` representa reconhecimento administrativo integral; o recebimento bruto soma os contratos atualmente pagos, sem representar receita líquida ou fluxo financeiro por data de pagamento.

O recorte exclui associação, lotes, comprovação documental, atividades, capacidade, benefícios, trabalhos, credenciamento, presença, certificação, reembolso e histórico operacional. Também não armazena parcelas, tentativas, meio ou data de pagamento. As regras de negócio e a matriz de mecanismos detalham as regras e o alcance de sua implementação.

2. Modelagem

2.1. Modelo conceitual

O Modelo Entidade-Relacionamento oficial adota a notação Peter Chen e contém cinco entidades, quatro relacionamentos e 24 atributos conceituais, incluindo cinco identificadores. A Figura 1.1 reproduz o artefato produzido no BrModelo. A especificação textual do MER complementa a leitura de entidades, atributos e cardinalidades.

Evento representa o evento de continuidade entre realizações, com identificador, nome e sigla opcional. **Edição** representa cada realização, com identificador, designação, datas inicial e final, modalidade, local condicionado e datas de abertura e fechamento das inscrições. **Participante** mantém identificador, nome completo, e-mail e CPF facultativo. **Categoria** mantém identificador, nome global único, descrição opcional e preço vigente. **Inscrição** representa a contratação, com identificador, data, situação, valor contratado e canal opcional.

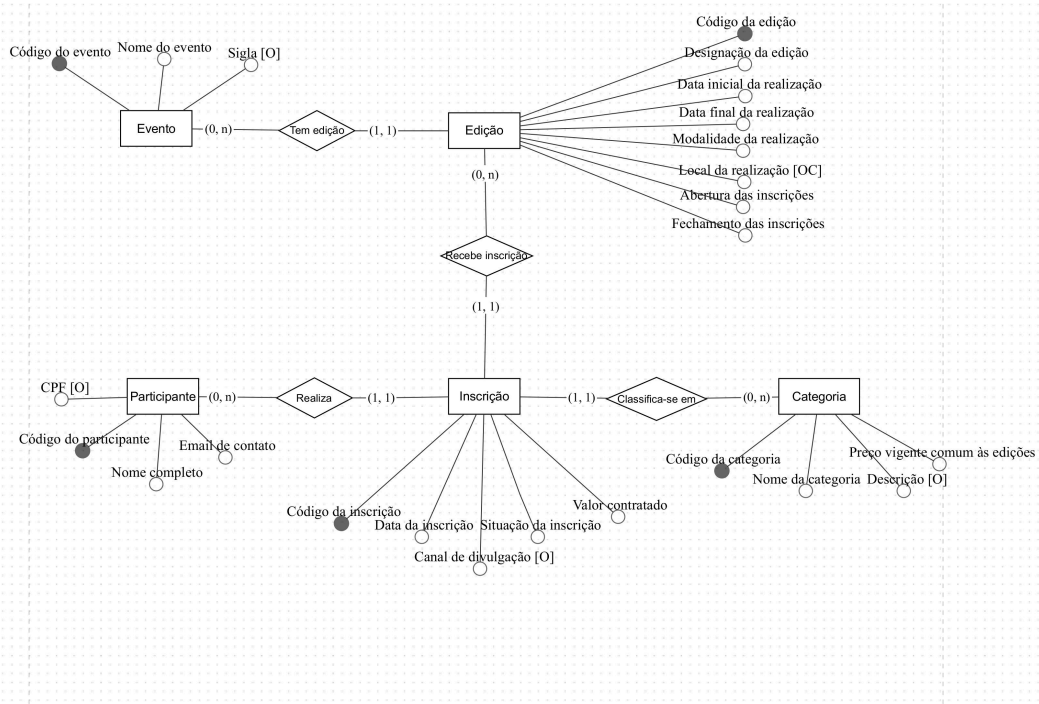


Figura 1.1. MER oficial em Peter Chen, com Inscrição central e Categoria global.

Evento se relaciona com Edição; Inscrição se relaciona diretamente com Edição, Participante e Categoria. Em cada relacionamento, o lado pai pode ter zero ou muitos filhos, enquanto o filho se vincula a exatamente um pai. Assim, uma edição tem um evento, e uma inscrição tem um participante, uma edição e uma categoria. O grafo de relacionamentos não contém ciclos. Não existe relacionamento direto entre Categoria e Edição.

2.2. Modelo lógico e domínios

Cada entidade origina uma tabela. Os relacionamentos de um para muitos são traduzidos por quatro chaves estrangeiras obrigatórias no lado filho. As cinco chaves primárias substitutas utilizam `BIGINT GENERATED BY DEFAULT AS IDENTITY`, sem significado de negócio. A Tabela 1.2 apresenta todas as 28 colunas do modelo lógico. A marca `?` indica coluna que aceita `NULL`; as demais são `NOT NULL`.

Tabela 1.2. Esquema relacional: colunas, nulabilidade e chaves.

Tabela	Colunas	Chaves
`evento`	`id_evento`, `nome`, `sigla?`	PK: `id_evento`
`edicao`	`id_edicao`, `id_evento`, `designacao`, `data_inicio`, `data_fim`, `modalidade`, `local_realizacao?`, `abertura_inscricoes`, `fechamento_inscricoes`	PK: `id_edicao`; FK: `id_evento` para `evento`
`participante`	`id_participante`, `nome_completo`, `email`, `cpf?`	PK: `id_participante`; UNIQUE: `cpf`
`categoria`	`id_categoria`, `nome`, `descricao?`, `preco_vigente`	PK: `id_categoria`; UNIQUE: `nome`
`inscricao`	`id_inscricao`, `id_participante`, `id_categoria`, `id_edicao`, `data_inscricao`, `situacao`, `valor_contratado`, `canal_divulgacao?`	PK: `id_inscricao`; três FK simples; UNIQUE: (`id_participante`, `id_edicao`)

São 23 colunas obrigatórias e cinco opcionais ou condicionadas: `sigla`, `local_realizacao`, `cpf`, `descricao` e `canal_divulgacao`. Identificadores e referências são `BIGINT`; datas utilizam `DATE`; preços e contratos usam `DECIMAL(12,2)`. Os textos utilizam `VARCHAR` com limites definidos no DDL. Situação e canal usam domínio textual limitado por `CHECK`, e `situacao` tem padrão `PENDENTE`.

As chaves estrangeiras simples são `edicao.id_evento`, `inscricao.id_participante`, `inscricao.id_categoria` e `inscricao.id_edicao`. Categoria não contém `id_edicao`, e o esquema não utiliza FK composta. A unicidade de (`id_participante`, `id_edicao`) impede uma segunda inscrição do mesmo cadastro na edição, independentemente da situação. As outras unicidades se aplicam ao CPF informado e ao nome global da categoria.

O preço vigente e o valor contratado representam fatos distintos. A cópia inicial é uma regra da operação de criação; não se exige igualdade permanente entre esses valores. A documentação de dependências funcionais apresenta a análise preliminar das dependências do esquema, e o registro de decisões explica as escolhas. Os tipos, tamanhos e constraints completos estão no Anexo A.

3. Implantação

3.1. Tradução do MER e integridade declarativa

A implantação utiliza PostgreSQL 17 e pgAdmin 4. O script DDL, reproduzido no Anexo A, cria primeiro Evento, depois Edição, Participante e Categoria, e finalmente Inscrição. A ordem atende às dependências das chaves estrangeiras. O script delimita uma transação e deve ser executado integralmente, uma vez, em base nova e vazia.

O mapeamento preserva a identidade de cada entidade como PK e os quatro relacionamentos por FK `NOT NULL`. As regras de identificação alternativas são implementadas pelas três restrições `UNIQUE`. O DDL contém **33 constraints nomeadas: cinco PK, três UNIQUE, quatro FK e 21 CHECK**. As 23 declarações `NOT NULL` são contabilizadas separadamente. Os `CHECK` verificam identificadores e valores positivos, textos não vazios, formato de CPF, domínios, ordem de datas e coerência entre modalidade e local.

Essas restrições impedem violações de domínio, unicidade, referências e condições da própria linha. Não verificam a janela da edição durante uma inserção em Inscrição, a cópia do preço vigente nem o histórico de transições de situação. O `DEFAULT 'PENDENTE'` somente fornece um valor quando ele é omitido; não impede uma inserção que informe explicitamente outro estado admitido.

3.2. Protocolo, acessos e consultas

O protocolo de operações controladas contém cinco atos independentes, executados em transações `SERIALIZABLE`: criar inscrição pendente com cópia do preço e verificação da janela (OC01), reconhecer quitação (OC02), cancelar pendente (OC03), reajustar o preço global (OC04) e alterar a janela somente antes de existirem inscrições (OC05). Cada operação exige conferir a quantidade de linhas afetadas e o resultado antes de concluir. Uma recusa ou erro demanda `ROLLBACK`; o erro de serialização `40001` demanda repetir a transação inteira.

O script de acessos define `eventos_operacao` e `eventos_consulta` como perfis de uso mediante concessões aos logins previamente criados. A operação recebe permissões por objeto e coluna; a consulta utiliza quatro views gerenciais: `vw_resumo_edicao`, `vw_resumo_categoria`, `vw_resumo_canal` e `vw_inscricoes_diarias`. Os relatórios do núcleo agregam os indicadores previstos no recorte.

Os privilégios reduzem a superfície de escrita, mas não obrigam a seguir todo o protocolo. O operador com `UPDATE(situacao)` pode realizar transições inadequadas usando SQL arbitrário, e o proprietário pode contornar restrições de acesso. As consultas de diagnóstico detectam anomalias, sem impedir sua gravação. A matriz de mecanismos distingue essas garantias; a entrega não utiliza os diagnósticos como substitutos de constraints ou de procedimentos.

3.3. População e resultados disponíveis

A carga determinística utiliza apenas dados fictícios. Sua execução prevê as quantidades da Tabela 1.3, descritas também na documentação da população. Edição, Participante e Inscrição atingem o mínimo de cinco mil registros em três tabelas. Os identificadores explícitos da carga são acompanhados de reinício das identidades para permitir inserções posteriores.

Tabela 1.3. Quantidades previstas pela carga SQL.

Tabela	Linhas previstas
Evento	500
Edição	5.000
Categoria	2
Participante	6.000
Inscrição	12.000
Total	23.502

Há confirmação de execução registrada no relatório técnico de testes, mas não existe exportação individual de contagens de todas as tabelas. Por isso, a Tabela 1.3 apresenta quantidades derivadas do SQL. A exportação V06 registra efetivamente os totais por situação da Tabela 1.4.

Tabela 1.4. Resultado capturado em V06: inscrições e valores contratados.

Situação	Quantidade	Total contratado (R$)
PAGA	4.001	500.120,00
PENDENTE	4.000	500.000,00
CANCELADA	3.999	499.850,00
Total	12.000	1.499.970,00

O recebimento bruto atual é apenas a soma dos contratos `PAGA`: **R$ 500.120,00**. Os totais pendente e cancelado não são recebimentos. A categoria Estudante é reajustada de R$ 100,00 para R$ 120,00; seis mil contratos antigos preservam R$ 100,00 e a inscrição 12000 contrata R$ 120,00. Esse cenário evidencia a distinção entre preço atual e contrato.

3.4. Verificação e limites da entrega

O roteiro reúne **78 blocos identificados**, incluindo os contextos Q00 e M00: dez consultas de estrutura, seis de população, seis verificações V, quatro diagnósticos, cinco relatórios, nove casos válidos, vinte inválidos, cinco testes de acesso, dois de concorrência e onze consultas de metadados. O relatório técnico distingue os casos com arquivos de evidência das execuções confirmadas sem captura individual.

Os registros disponíveis de casos inválidos mostram rejeições por PK, FK, UNIQUE, nulabilidade e CHECK. TC01 testa a unicidade participante/edição e possui confirmação sem captura individual. TC02 registra espera, erro de serialização `40001`, repetição que atualiza zero linhas e limpeza após disputa entre quitação e cancelamento. Esses resultados demonstram os cenários registrados, sem provar a segurança de toda escrita direta.

As exportações M00 a M09 foram comparadas ao DDL e sustentam cinco tabelas, 28 colunas e as 33 constraints nomeadas, conforme o resumo de metadados.

M10 contém uma lista anterior de nomes de views, sem comprovar as definições atuais. As quatro views são descritas pelo script vigente. As cinco PK e três UNIQUE implicam oito estruturas de índice, sem índices adicionais declarados, benchmark ou inventário físico independente; o alcance está no registro de índices.

O Marco 1 entrega um núcleo gerencial reproduzível por SQL, com modelagem, estrutura, carga e verificações. A rastreabilidade conecta requisitos, regras, mecanismos e evidências. Reconstruir mudanças de situação, pagamentos por data ou atos administrativos exigiria ampliar o modelo. O uso correto depende tanto das restrições declarativas quanto do protocolo operacional e de seus limites documentados.

4. Anexos

Os anexos reúnem o DDL integral, o tutorial e o índice dos códigos complementares. Os links são relativos ao diretório deste relatório e apontam para materiais existentes no próprio projeto; os caminhos de scripts citados no tutorial partem da raiz do repositório. Para reproduzir a entrega, mantenha a estrutura de pastas.

Anexo A. Script DDL integral

Fonte: sql/01_estrutura.sql. A listagem abaixo reproduz integralmente o arquivo vigente; apenas as quebras visuais de linhas longas podem variar no PDF.

```
-- EventOS: MER oficial de cinco entidades com Inscrição central.
-- Categoria é catálogo global, com preço vigente comum a todas as edições.
-- Executar integralmente numa base limpa.

START TRANSACTION;

CREATE TABLE evento (
    id_evento BIGINT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
    nome VARCHAR(200) NOT NULL,
    sigla VARCHAR(30),
    CONSTRAINT pk_evento PRIMARY KEY (id_evento),
    CONSTRAINT ck_evento_id CHECK (id_evento > 0),
    CONSTRAINT ck_evento_nome CHECK (CHAR_LENGTH(TRIM(nome)) > 0),
    CONSTRAINT ck_evento_sigla CHECK (sigla IS NULL OR CHAR_LENGTH(TRIM(sigla)) > 0)
);

CREATE TABLE edicao (
    id_edicao BIGINT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
    id_evento BIGINT NOT NULL,
    designacao VARCHAR(60) NOT NULL,
    data_inicio DATE NOT NULL,
    data_fim DATE NOT NULL,
    modalidade VARCHAR(10) NOT NULL,
    local_realizacao VARCHAR(200),
    abertura_inscricoes DATE NOT NULL,
    fechamento_inscricoes DATE NOT NULL,
    CONSTRAINT pk_edicao PRIMARY KEY (id_edicao),
    CONSTRAINT ck_edicao_id CHECK (id_edicao > 0),
    CONSTRAINT fk_edicao_evento FOREIGN KEY (id_evento) REFERENCES evento (id_evento),
    CONSTRAINT ck_edicao_designacao CHECK (CHAR_LENGTH(TRIM(designacao)) > 0),
    CONSTRAINT ck_edicao_realizacao CHECK (data_inicio <= data_fim),
    CONSTRAINT ck_edicao_janela CHECK (abertura_inscricoes <= fechamento_inscricoes AND
        fechamento_inscricoes <= data_fim),
    CONSTRAINT ck_edicao_modalidade CHECK (modalidade IN ('PRESENCIAL', 'HIBRIDA', 'REMOTA')),
    CONSTRAINT ck_edicao_local CHECK (modalidade NOT IN ('PRESENCIAL', 'HIBRIDA', 'REMOTA') OR
        (modalidade = 'REMOTA' AND local_realizacao IS NULL) OR (modalidade IN ('PRESENCIAL',
        'HIBRIDA') AND local_realizacao IS NOT NULL AND CHAR_LENGTH(TRIM(local_realizacao)) >
        0))
);
```

```
CREATE TABLE participante (
    id_participante BIGINT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
    nome_completo VARCHAR(150) NOT NULL,
    email VARCHAR(254) NOT NULL,
    cpf VARCHAR(11),
    CONSTRAINT pk_participante PRIMARY KEY (id_participante),
    CONSTRAINT ck_participante_id CHECK (id_participante > 0),
    CONSTRAINT uq_participante_cpf UNIQUE (cpf),
    CONSTRAINT ck_participante_nome CHECK (CHAR_LENGTH(TRIM(nome_completo)) > 0),
    CONSTRAINT ck_participante_email CHECK (CHAR_LENGTH(TRIM(email)) > 0),
    CONSTRAINT ck_participante_cpf CHECK (cpf IS NULL OR cpf SIMILAR TO '[0-9]{11}')
);

CREATE TABLE categoria (
    id_categoria BIGINT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
    nome VARCHAR(80) NOT NULL,
    descricao VARCHAR(300),
    preco_vigente DECIMAL(12,2) NOT NULL,
    CONSTRAINT pk_categoria PRIMARY KEY (id_categoria),
    CONSTRAINT ck_categoria_id CHECK (id_categoria > 0),
    CONSTRAINT uq_categoria_nome UNIQUE (nome),
    CONSTRAINT ck_categoria_nome CHECK (CHAR_LENGTH(TRIM(nome)) > 0),
    CONSTRAINT ck_categoria_descricao CHECK (descricao IS NULL OR CHAR_LENGTH(TRIM(descricao))
        > 0),
    CONSTRAINT ck_categoria_preco CHECK (preco_vigente > 0)
);

CREATE TABLE inscricao (
    id_inscricao BIGINT GENERATED BY DEFAULT AS IDENTITY NOT NULL,
    id_participante BIGINT NOT NULL,
    id_categoria BIGINT NOT NULL,
    id_edicao BIGINT NOT NULL,
    data_inscricao DATE NOT NULL,
    situacao VARCHAR(9) DEFAULT 'PENDENTE' NOT NULL,
    valor_contratado DECIMAL(12,2) NOT NULL,
    canal_divulgacao VARCHAR(11),
    CONSTRAINT pk_inscricao PRIMARY KEY (id_inscricao),
    CONSTRAINT ck_inscricao_id CHECK (id_inscricao > 0),
    CONSTRAINT fk_inscricao_participante FOREIGN KEY (id_participante) REFERENCES participante
        (id_participante),
    CONSTRAINT fk_inscricao_categoria FOREIGN KEY (id_categoria) REFERENCES categoria
        (id_categoria),
    CONSTRAINT fk_inscricao_edicao FOREIGN KEY (id_edicao) REFERENCES edicao (id_edicao),
    CONSTRAINT uq_inscricao_participante_edicao UNIQUE (id_participante, id_edicao),
    CONSTRAINT ck_inscricao_situacao CHECK (situacao IN ('PENDENTE', 'PAGA', 'CANCELADA')),
    CONSTRAINT ck_inscricao_valor CHECK (valor_contratado > 0),
    CONSTRAINT ck_inscricao_canal CHECK (canal_divulgacao IS NULL OR canal_divulgacao IN
        ('SITE_EVENTO', 'REDE_SOCIAL', 'EMAIL', 'INDICACAO', 'OUTRO'))
);

COMMIT;
```

Anexo B. Tutorial de reprodução

O ambiente documentado utiliza PostgreSQL 17 e pgAdmin 4, sem linguagens externas de conexão. O README do projeto e a ordem de execução detalham o procedimento. Os resultados existentes foram consultados para este relatório; o banco não foi reexecutado durante sua redação.

1. Criar pelo pgAdmin uma base nova e vazia, UTF-8, por exemplo `eventos_entrega`. Abrir o Query Tool como proprietário da base, utilizando o esquema `public` e Auto-commit habilitado. Não aplicar o DDL sobre tabelas existentes.

2. Executar `sql/01_estrutura.sql` inteiro, uma vez. Conferir o `COMMIT` sem erro e a criação das cinco tabelas.

3. Executar Q00 a Q09 de `sql/verificacoes_estrutura.sql`, consulta por consulta. Q01, Q02 e Q04 devem retornar zero linhas. Conferir 28 colunas, 23 obrigatórias, cinco opcionais, cinco identidades e 33 constraints nomeadas.

4. Executar `sql/02_carga.sql` inteiro, uma vez. Executar `sql/contagens.sql` e P00 a P05 de `sql/verificacoes_populacao.sql`. Conferir as quantidades da Tabela 1.3; P01 deve retornar zero linhas.

5. Executar V01 a V06 de `sql/04_verificacoes.sql` e DG01 a DG04 de `sql/diagnosticos.sql`. V01 a V04 e DG01 a DG04 devem retornar zero linhas. V05 deve mostrar seis contratos representativos: cinco de R$ 100,00 e um de R$ 120,00, com preço vigente de R$ 120,00. Conferir V06 com a Tabela 1.4.

6. Criar ou conferir os logins `eventos_operacao` e `eventos_consulta` pelo pgAdmin, com senhas locais. Manter desabilitados Superuser, Create databases, Create roles e Replication e evitar filiação privilegiada. Se necessário, conceder CONNECT na base e USAGE em `public`; não conceder CREATE no esquema nem privilégios gerais. Senhas não integram o pacote.

7. Como proprietário, executar `sql/acessos.sql` inteiro para criar quatro views e conceder os privilégios previstos. Executar R01 a R05 de `sql/relatorios_nucleo.sql`; R01 prevê 5.000 edições, R02 duas categorias e o bruto previsto é R$ 500.120,00.

8. Executar TV01 a TV09 de `testes/casos_validos.sql`, um caso por vez, respeitando a ordem: transação, alterações, consultas, `ROLLBACK` e consultas de pós-condição. Salvar os resultados antes da reversão quando indicado.

9. Executar TI01 a TI20 de `testes/casos_invalidos.sql` isoladamente. Selecionar a abertura da transação e o comando inválido, registrar o erro, executar `ROLLBACK` em separado e depois consultar a pós-condição. Não executar o arquivo inteiro de uma vez.

10. Executar PA01 a PA05 de `testes/acessos.sql` autenticado no perfil indicado: consulta em PA01 e PA04; operação em PA02, PA03 e PA05. Abrir conexões com esses logins; renomear uma aba não muda o usuário. Após erro, executar `ROLLBACK` separado.

11. Como proprietário, executar TC01 e TC02 de `testes/concorrencia.sql` em duas sessões independentes A e B, alternando os trechos conforme os comentários. Registrar os resultados e realizar a limpeza. Após `40001`, desfazer a transação abortada e repetir integralmente a transação indicada.

12. Como proprietário, executar M00 a M10 de `sql/05_metadados.sql`, consulta por consulta. M10 depende da criação das views. Exportar os resultados, preservando identificação da consulta e perfil usado.
13. Repetir contagens, situações, canais e diagnósticos. Não devem restar inscrições de teste 20001 ou 20002; Categoria 1 deve custar R$ 120,00, Inscrição 2 deve estar PENDENTE e os totais por situação devem coincidir com a Tabela 1.4.

`sql/operacoes_controladas.sql` contém operações administrativas independentes com parâmetros de demonstração e `COMMIT`. Não executar todos os blocos juntos como teste: isso altera a carga de referência. Os casos válidos demonstram as operações com reversão. Os diagnósticos devem ser usados para detectar estados anômalos, sem atribuir a eles impedimento de gravação.

Anexo C. Índice dos códigos complementares

O DDL está integralmente no Anexo A. Os quatorze arquivos SQL restantes são relacionados abaixo e acompanham o relatório na estrutura original do projeto. Carga e testes extensos são mantidos em seus arquivos executáveis, evitando divergência entre transcrição e código.

Tabela 1.5. Códigos complementares e forma de execução.

Arquivo	Finalidade e execução
sql/02_carga.sql	Carga determinística; integral, uma vez, após o DDL.
sql/contagens.sql	Contagens por tabela.
sql/verificacoes_estrutura.sql	Q00 a Q09; por consulta.
sql/verificacoes_populacao.sql	P00 a P05; por consulta.
sql/04_verificacoes.sql	V01 a V06; por consulta.
sql/diagnosticos.sql	DG01 a DG04; diagnóstico, sem impedir escrita.
sql/acessos.sql	Views e privilégios; integral após criar os logins.
sql/relatorios_nucleo.sql	R01 a R05; relatórios gerenciais.
sql/operacoes_controladas.sql	OC01 a OC05; atos independentes, nunca execução integral conjunta.
sql/05_metadados.sql	M00 a M10; por consulta, como proprietário.
testes/casos_validos.sql	TV01 a TV09; um caso de cada vez, com reversão.
testes/casos_invalidos.sql	TI01 a TI20; isolados, com erro e ROLLBACK separado.
testes/acessos.sql	PA01 a PA05; com o login exigido em cada caso.
testes/concorrencia.sql	TC01 e TC02; duas sessões independentes e limpeza final.

Anexo D. Documentação e evidências de apoio

- Escopo e requisitos, regras de negócio e propostas de extensão: delimitação do minimundo.
- MER oficial, especificação conceitual e modelo lógico: artefatos de modelagem.
- Matriz de mecanismos, decisões, dependências funcionais e rastreabilidade: justificativas e alcance das regras.
- População, resumo de metadados e índices: quantidades e verificações estruturais.
- Relatório técnico de testes, índice de evidências e resultado V06: resultados e limites de comprovação.
- README e ordem de execução: reprodução do pacote.